

BENNETT UNIVERSITY

School of Computer Science Engineering and Technology

Computational Thinking and Programming

Course Code: 2026CSAE100

PYTHON PROJECT REPORT

Personal Finance and Investment System

A menu-driven stock, portfolio, and expense tracker

Student Name	Aniket Singh - project idea proposer; requirements and project documentation
Enrollment Number	To be filled
Batch / Section	To be filled
Team Member 2	Utkarsh Vijay - Project Leader; stock data, APIs, analysis, calculations, charts, and UI integration
Team Member 3	Parth Mishra - stock and investment fundamentals
Team Member 4	Kumar Udayaditya Prakash - stock analysis concepts
Team Member 5	Md Hossen Shahin Rana - Expense Tracker

1 Problem Statement

Users who are learning about investing and managing personal finances often switch between tools to review company information, track holdings and trades, and understand daily spending. This project combines these tasks in one menu-driven Python program. It supports basic market review and personal record keeping; it is an educational tracker, not investment advice.

2 Project Objectives

The project aims to:

1. Provide one console menu for stock analysis, portfolio tracking, and expense management.
2. Retrieve available company information and historical prices for a ticker symbol.
3. Track holdings and trades, including market value and realized or unrealized profit/loss.
4. Record, find, edit, and summarize expenses with monthly threshold alerts.
5. Show useful summaries and a one-year closing-price chart when data is available.

3 Main Features / Modules

The project contains the following features:

S. No.	Module / Feature	Description
1	Share Price and Fundamental Analyzer	Shows company details, selected financial measures, statements, recent prices, and a one-year closing-price chart.
2	Stock Portfolio and Trading Tracker	Adds holdings, estimates current value and unrealized P/L, records sales and realized P/L, and summarizes performance and sectors.
3	Daily Expense and Threshold Tracker	Records, searches, edits, and summarizes expenses; supports thresholds, category analytics, savings, cash flow, and recurring expenses.
4	Main Menu / Console Interface	Routes the user between the three modules and returns to the menu until exit is selected.
5	Input Validation and Error Handling	Checks dates and numeric inputs, rejects invalid trade quantities, and handles missing data or request errors in selected paths.

4 Algorithm / Flowchart / Program Design

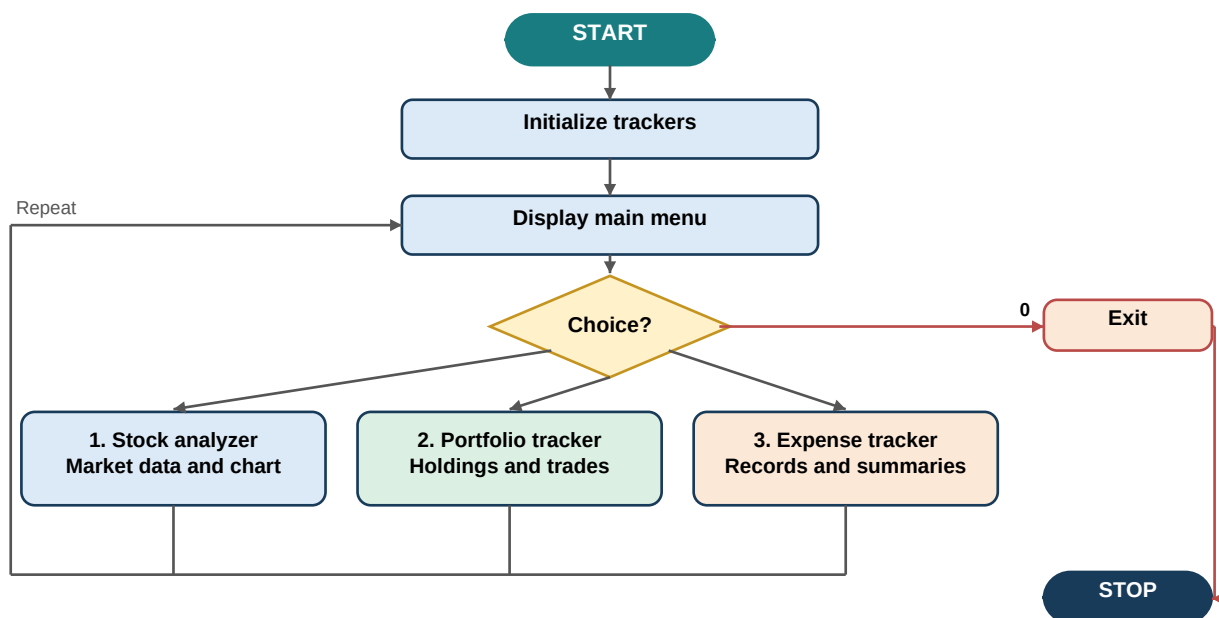
The program starts with a main menu and routes the selected option to one of three modules. Each module processes input and returns to its menu; the main loop continues until the user exits.

Algorithm / Pseudocode

1. Start.
2. Create the portfolio and expense trackers; display the main menu.
3. If Stock Analyzer is selected, read a ticker, request available company and price data, and display selected information or a chart.
4. If Portfolio or Expense Tracker is selected, process its menu actions, validate inputs, update in-memory records, calculate summaries, and return to the menu.
5. For an invalid choice, display an error and show the menu again. For choice 0, stop.

Flowchart / Design

The flowchart shows the main program path and its three menu branches.



5 Python Concepts Used

Mention the Python concepts used in your project.

Python Concept	Where / How It Is Used
Variables and Data Types	Ticker/category strings, numeric share and price values, dates, and recurring-expense flags.
Conditional Statements	Menu routing, input checks, threshold alerts, empty-data checks, and sale-quantity validation.
Loops	Main and module menus repeat; loops process holdings, trades, expenses, and summary records.
Lists / Tuples / Sets / Dictionaries	Holdings, trades, and expenses use lists of dictionaries; dictionaries aggregate sector and category totals.
Functions	Methods and classes organize the analyzer, portfolio tracker, and expense tracker behaviors.
File Handling	Not implemented; portfolio and expense records remain in memory for the current run.
Exception Handling	Handles invalid numeric/date input and selected failures while requesting market data.
Libraries Used, if any	datetime/date, yfinance, and matplotlib.pyplot. pandas is imported, but the listing does not call it directly.

6 Computational Feature

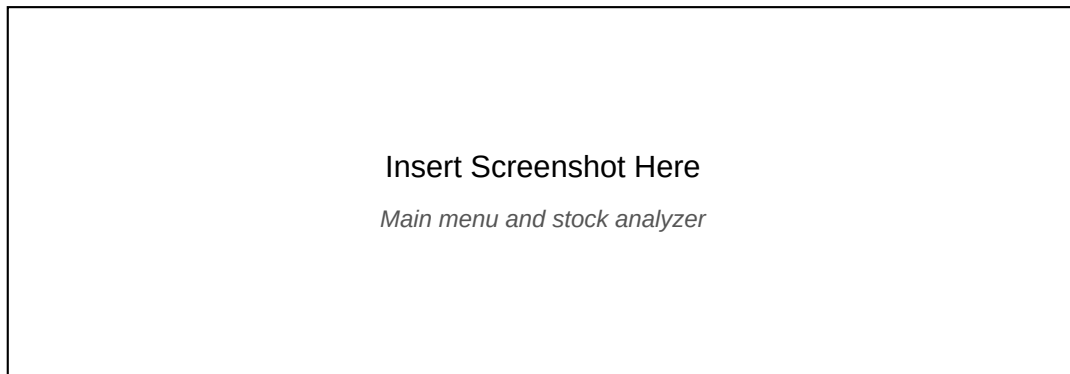
The program summarizes market and personal-finance records. For each holding, cost basis = shares x purchase price; estimated market value = shares x current price; unrealized P/L = market value - cost basis; percentage return = (P/L / cost basis) x 100 when cost basis is positive. Realized P/L on a sale equals proceeds minus the recorded cost of the shares sold.

The portfolio view adds holding values and compares returns to identify the best and worst performers. The expense tracker totals spending by month and category. A set monthly threshold triggers a warning at 80% and an exceeded status at 100% or more. The analyzer plots one year of closing prices when historical data is available.

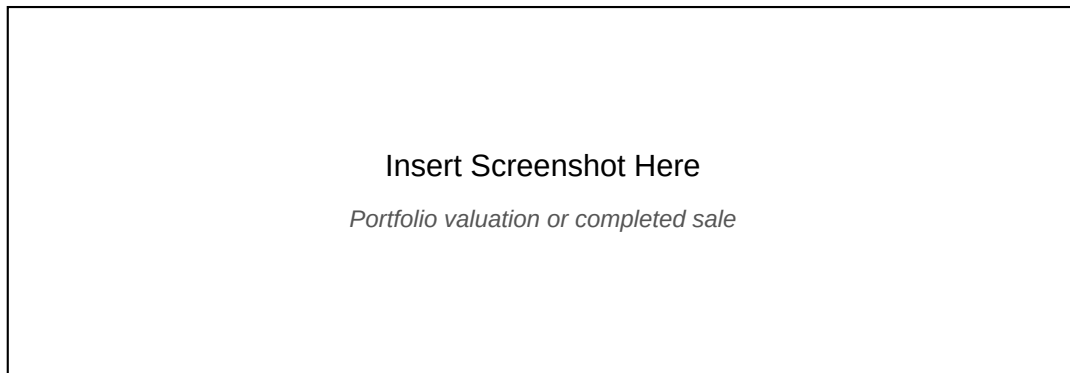
7 Important Outputs / Screenshots

Insert screenshots showing the important parts of the working project.

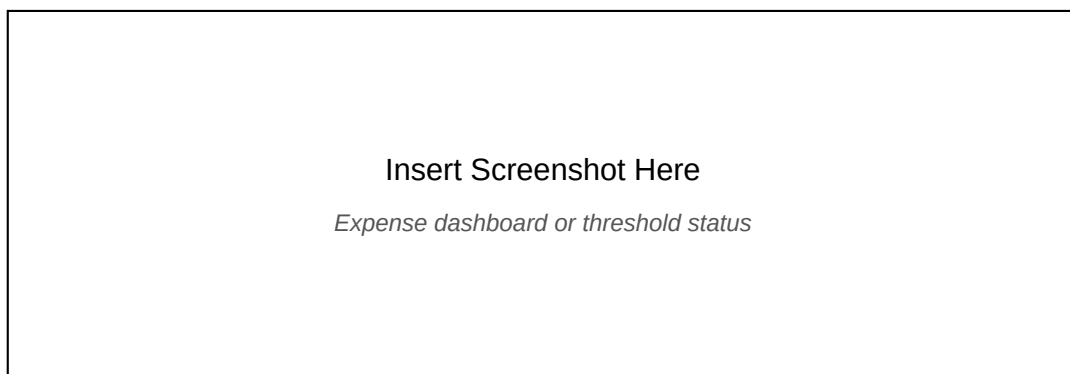
Screenshot 1



Screenshot 2



Screenshot 3



8 Testing

Test your project using different types of inputs.

Test	Input	Expected Output	Actual Output	Result
1	Valid ticker and positive holding values.	Holding is added and shown in portfolio valuation.	Pending execution	Pending
2	Sell fewer shares than available.	Trade is recorded; holding quantity and realized P/L update.	Pending execution	Pending
3	Sell exactly all shares in a holding.	Holding reaches zero and is removed from the watchlist.	Pending execution	Pending
4	Enter a non-numeric amount or share quantity.	Input error is handled and an error message appears.	Pending execution	Pending
5	Expense date blank, then invalid date.	Blank uses today; invalid date is rejected and requested again.	Pending execution	Pending

Include at least:

- one normal test case;
- one boundary or special test case; and
- one invalid input test case.

9 Team Member Contributions

Clearly mention the contribution of each team member.

Student Name	Enrollment No.	Contribution
Aniket Singh	To be filled	Proposed the project idea; contributes to requirements and project documentation.
Utkarsh Vijay	To be filled	Project Leader; leads coordination and integration, including market data/API retrieval, metrics, charting, portfolio calculations, and UI connection.
Parth Mishra	To be filled	Covers stock and investment fundamentals such as shares, tickers, price, market cap, OHLCV, portfolios, and returns.
Kumar Udayaditya Prakash	To be filled	Covers fundamental and technical stock analysis; the code displays selected financial measures and moving averages.
Md Hossen Shahin Rana	To be filled	Expense Tracker: records, validation, search/edit/delete, thresholds, category analytics, savings, cash flow, recurring expenses.

10 Challenges Faced and Learning

Briefly mention:

- Missing market fields or failed requests are handled in selected paths with price fallbacks, empty-data checks, and exception handling.
- Invalid values and dates are handled by validation and retry/error messages; persistent storage remains a future improvement.
- The project practices classes, menu loops, dictionaries, financial calculations, external data retrieval, charts, and expense aggregation.

11 Conclusion and Future Improvements

The program combines a stock analyzer, a portfolio and trading tracker, and a daily expense tracker behind one console menu. It retrieves available market data, calculates basic holding and trade results, summarizes expenses, and plots historical closing prices when data is available.

Possible improvements include saving records to a file or database and adding tested technical indicators such as RSI or MACD. Currency-aware display and an exportable report would also improve usability.

12 References / Acknowledgements

Mention any websites, documentation, datasets, tutorials, libraries, or AI tools consulted during the project.

1. Project source listing supplied by the team: Project2.0.pdf.
2. Report structure supplied by the team: Project Report Format.pdf.
3. yfinance API reference: <https://ranaroussi.github.io/yfinance/reference/index.html>.
4. pandas documentation: <https://pandas.pydata.org/docs/>.
5. Matplotlib documentation: <https://matplotlib.org/stable/>.
6. OpenAI Codex assisted in drafting this report from the supplied materials; the team should review the final content.

Declaration:

We confirm that we understand the submitted project and have acknowledged the external resources used during development.